

RAPPORT SAE DEV1.1

Table des matières :

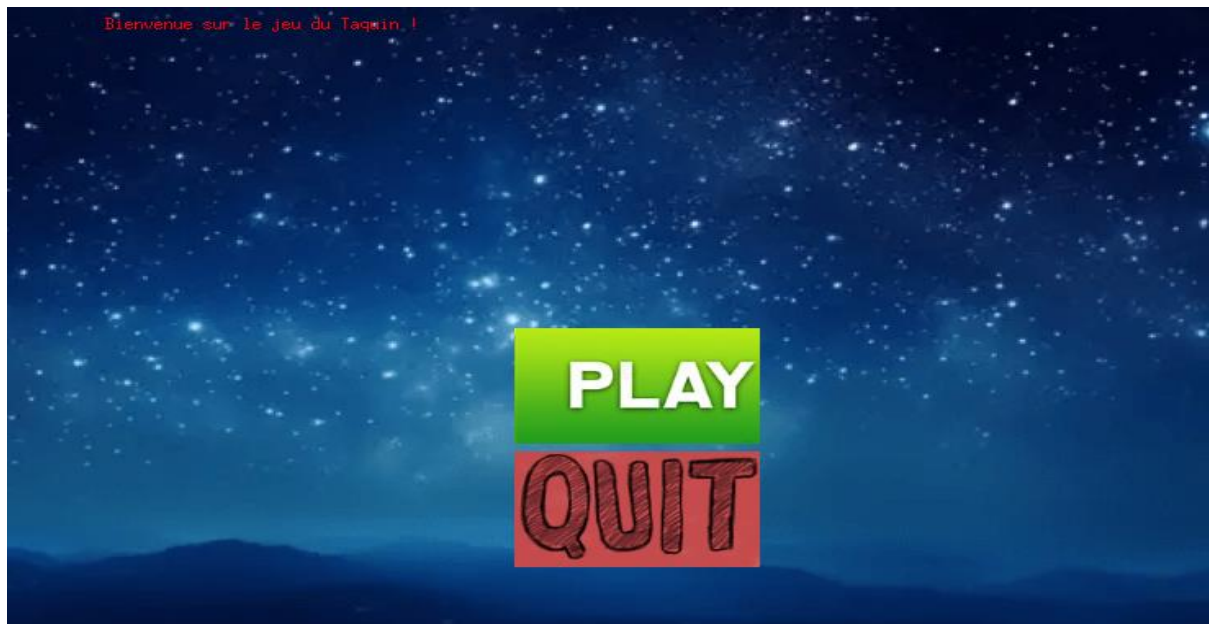
- Description des fonctionnalités de notre programme
- Présentation de la structure du programme, y compris une justification du découpage en différents fichiers source, avec diagrammes à l'appui.
- Pourquoi nous avons découpé les fichiers comme ça ?
- SIMULATION D'UNE PARTIE ET DE C'EST VALEUR STOCKE
- Explications algorithmes de mélanges
- Conclusion personnelle

INTRODUCTION :

Lors de cette SAE nous devions à l'aide de la bibliothèque graphique "graph.h" mis à disposition par nos professeurs et celles que nous avons utilisées durant ce premier semestre reproduire le jeu du "Taquin" qui consiste à choisir une image parmi une sélection donnée, puis sélectionner la difficulté avec laquelle nous voulons jouer qui est représenté par le découpage de cette image(3x3,4x4,8x5). Après avoir sélectionnés le découpage voulu le puzzle est formés puis mélangé, c'est ensuite au joueur à l'aides des flèches directionnelles du clavier ou avec la souris de réussir à remettre le puzzle en ordre pour que l'on retombe sur l'images initiales. Bien évidemment le but du jeu est de réussir à le reconstituées en moins de coup possible.

Description des fonctionnalités de notre programme :

Pour répondre aux attentes du sujet nous avons mis en place un écran d'accueil dès le lancement de notre jeu car dans tous les jeux auquel nous avons jouées l'écran d'accueil était la première chose qui apparaissait sur notre écran.



Puis une fois que l'utilisateur a choisi s'il voulait jouer en cliquant sur le bouton joué une sélection de 3 images apparaît devant lui, il n'a plus qu'à choisir une image parmi les 3. Nous avons décidé de rester sur le système du clic sur l'images pour la choisir car cela était la

chose la plus intuitif selon nous, pas besoins de chercher comment choisir l'image il suffit juste de cliquer dessus.



Une fois que l'utilisateur a choisi l'image avec laquelle il veut jouer, il est redirigé vers le choix de découpage du puzzle qu'il devra reconstituer. Une colonne pour le découpage en lignes, et une autre ligne pour le découpage en colonne permettant à l'utilisateur de choisir

indépendamment le découpage de son puzzle est ainsi lui permettre une montée en difficulté progressive. Une fois les dimensions choisies il lui suffira de cliquer sur valider. Pour le côté pratique nous avons également mis à disposition une possibilité de retour vers le choix de l'images avec laquelle l'utilisateur souhaite ce challenger, car il est possible que dans la précipitation ou inversement par inattentions nous voulions choisir une image mais nous en choisissons une autre.

Choisissez les lignes et les colonnes :

Lignes :

3	4	5	6	7	8
---	---	---	---	---	---

Colonnes :

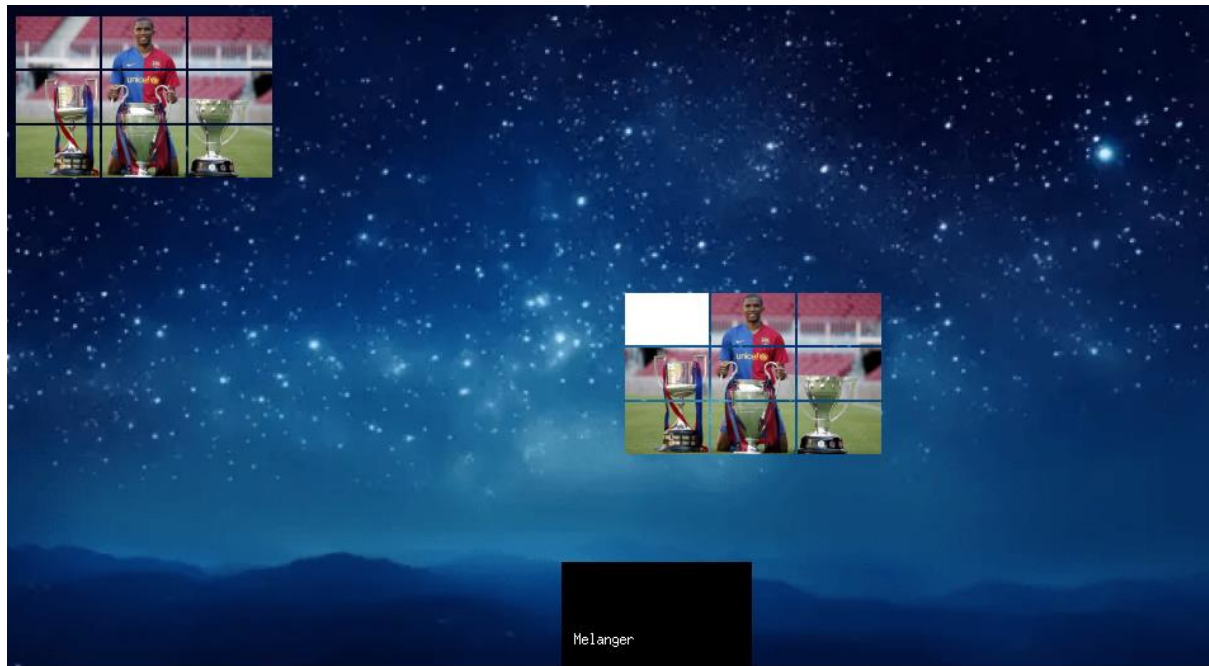
3	4	5	6	7	8
---	---	---	---	---	---

VALIDER



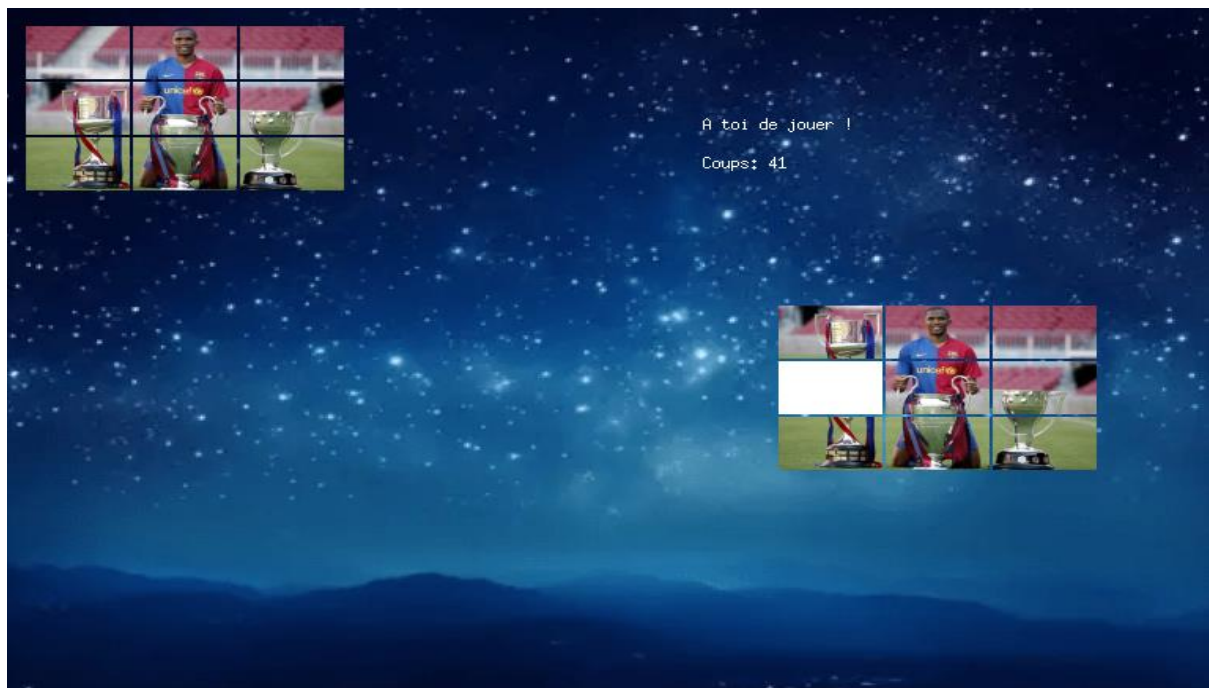
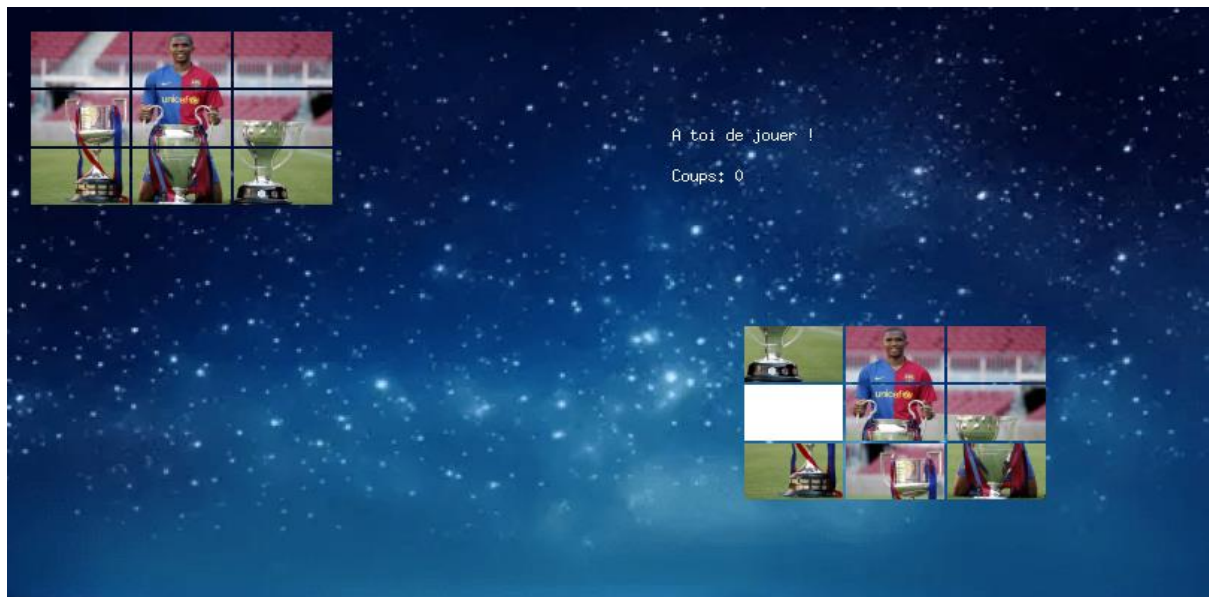
L'utilisateur est ensuite redirigé vers un écran où son image est sous formes de puzzle au centre de l'écran avec la première pièce du puzzle en partant de la gauche en blanc et

également en haut à gauche de l'écran cette fois si sans pièces blanches permettant à l'utilisateur de comprendre que ceci lui servira de modèle. En dessous du puzzle au centres se trouve un bouton "mélanger" qui lancera ensuite le jeu une fois qu'on clique dessus.

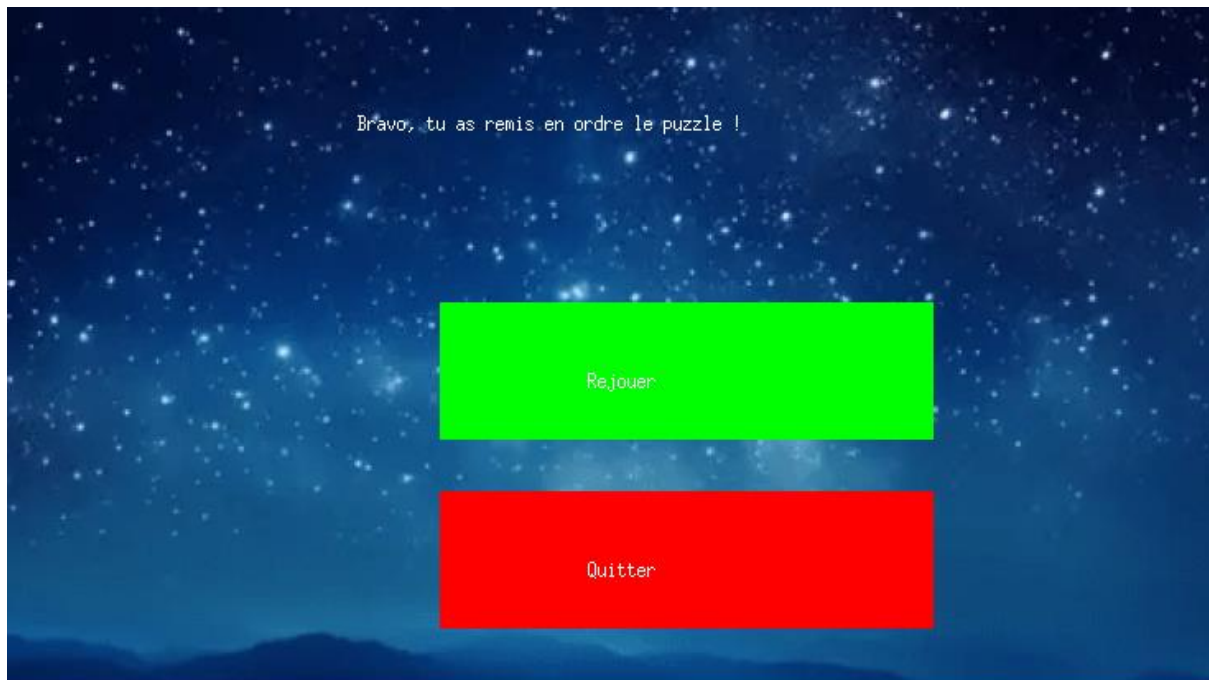


Une fois le jeu lancé il sera face à un puzzle en haut à gauche non mélangé qui lui servira de modèle pour reconstituer l'image originale, et au centre se trouve le puzzle mélangé que l'utilisateur doit remettre en ordre dans un premier temps, puis une fois que ce dernier

sera un peu plus à l'aise avec le jeu il pourra se lancer le défi de réussir en moins de coup possible. Ces derniers ont affiché à côté du puzzle.



Le puzzles remis en ordre déclenchera l'affichage d'écran de victoires où le joueur pourras choisir de quitter le jeu ou rejouer avec une image différente.



Présentation de la structure du programme, y compris une justification du découpage en différents fichiers source, avec diagrammes à l'appui.

Nous avons décidé d'organiser notre code à l'image d'une arborescence avec pour racines notre fonction `main()` qui se charge d'afficher l'écran d'accueil et son interface graphique dès

lors que l'utilisateur aura cliquer quelque part sur l'écran la fonction main() appellera la fonction Bouton1P_or_Q() qui vérifiera si c'est le bouton jouer ou cliquer, une fois que la zones cliquer représentera celle du bouton jouer main() appellera la fonction Choix_images() qui se contentera d'afficher les 3 images que l'utilisateur pourra choisir pour jouer. Une fois les images afficher c'est la fonction bouton_choix_images() qui sera appelé pour vérifier quelle image a été choisie.

A partir d'ici c'est bouton_Choix_image() qui prend le relais.

Cette dernière appellera 3 autres fonctions

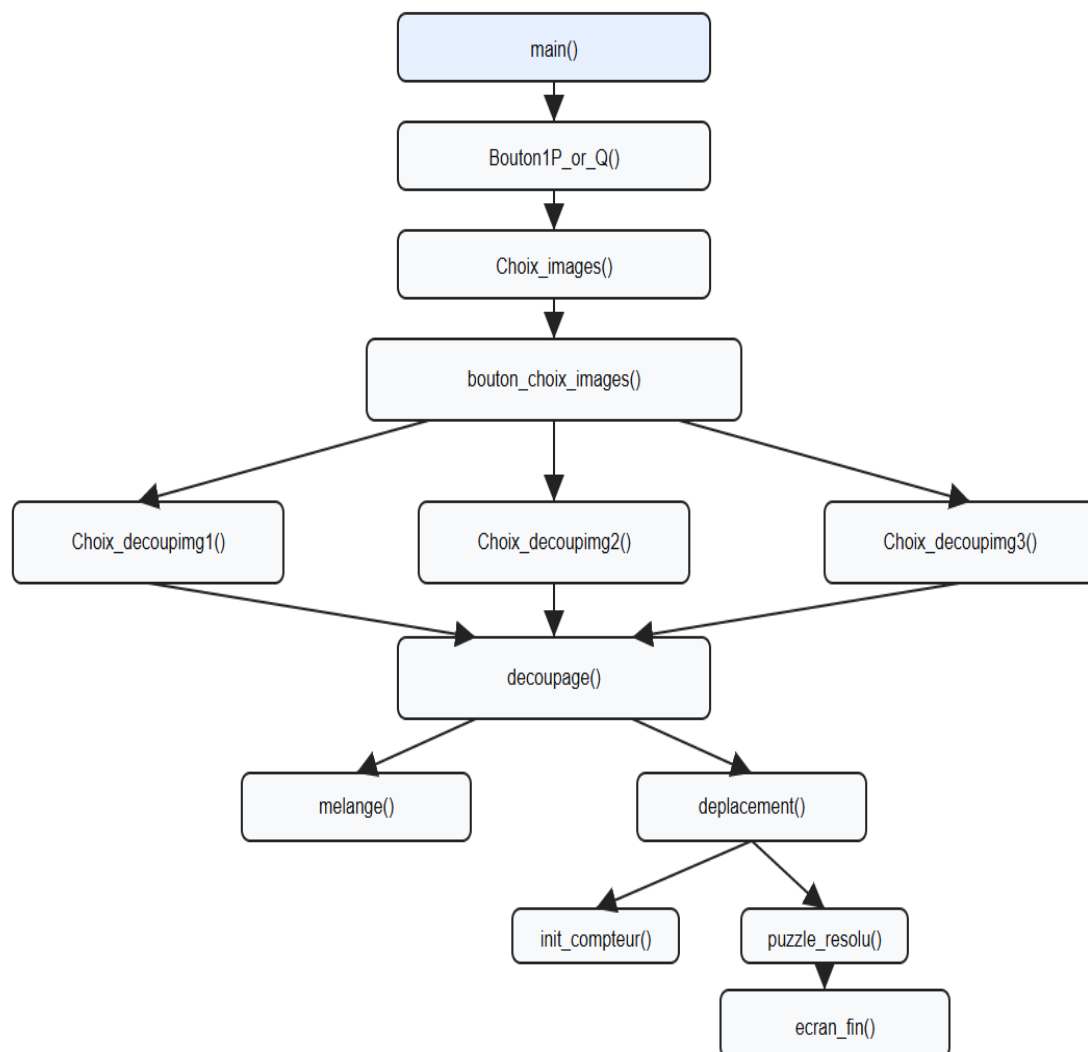
Choix_decoupimg1/Choix_decoupimg2/Choix_decoupimg3 qui correspondent toutes les 3 à une image différente. Une fois l'image choisie l'une des 3 fonction se déclenchera pour lancer l'écran du choix du découpage et attendra que chaque argument requis pour la fonction découpage soit sélectionné par l'utilisateur. Après ça elle attendra que l'utilisateur clique sur "valider" et c'est une fois toutes ces conditions réunies que Choix_decoupimg1/ Choix_decoupimg2/Choix_decoupimg3 appellera la fonction découpage.

La fonction découpage comme l'indique son nom va se charger de découper l'image en tuiles en fonction du choix de l'utilisateur, puis elle affichera sur l'écran l'image découpé qui servira de modèle au joueur en haut à gauche de l'écran et le puzzle avec lequel il jouera au centre. Si l'utilisateur clique sur le bouton "mélanger" alors la fonction découpage appellera la fonction mélange qui se chargera de mélanger le puzzle du joueur et enfin une fois le mélange fini la fonction découpage appellera la fonction déplacement qui aura pour mission d'autoriser le déplacement de pièces de puzzle uniquement si ces dernières sont adjacentes à la pièce de puzzle blanche et afficher le compteur qui représente le nombre de coups joués par l'utilisateur avec l'aide de la fonction init_compteur.

Si le puzzle est résolu alors la fonction déplacement appellera la fonction ecran_fin qui affichera le choix de soit rejouer soit quitter le jeu.

Représentation de la structure d'appel de fonction du programme

Arbre d'appels - Taquin (fonction → fonction)



Pourquoi nous avons découpé les fichiers comme ça ?

Nous avons choisi de découper les fichiers comme ça afin d'éviter que tous les fichiers dépendent les uns des autres.

En effet quand nous regardons quel fichier dépend de qui nous pouvons voir que :

Ceux dont main.o dépend : Bouton.h --> Choix_image.h --> bouton_Choix_image.h

Ceux dont Bouton.o dépend : Choix_image.h

Choix_image.h ne dépend d'aucun .h

Bouton_Choix_image.o : Choix_decoupimg1.h --> Choix_decoupimg2.h --> Choix_decoupimg3.h

Choix_decoupimg1.o/ Choix_decoupimg2.o/Choix_decoupimg3.o dépendent de : decoupage.h--> Choix_image.h --> bouton_Choix_image.h

Ceux dont decoupage.o dépend : melange.h --> deplacement.h

Melange.o ne dépend d'aucun .h

Ce dont deplacement.o dépend : ecranGagner.h --> puzzleResolu.h --> compteur.h

PuzzleResolu.h ne dépend d'aucun .h

Ce dont ecranGagner.h dépend : bouton_Choix_image.h --> Choix_image.h

Nous avons essayé de faire en sorte que chaque fonction ne dépende uniquement que de celle qui l'aide à effectuer la tâche qu'elle doit accomplir. Ce qui implique également que si on fait une modification dans un des fichiers nous ne sommes pas obligés de modifier tous les fichiers.

SIMULATION D'UNE PARTIE ET DE C'EST VALEUR STOCKE

État d'une partie en cours

Lorsque l'utilisateur va lancer le programme, il verra un écran d'accueil avec un bouton jouer et un bouton quitter sur lesquels il devra appuyer selon s'il veut jouer ou non.

Boutons de l'écran d'accueil

Le programme va donc lors de cette étape vérifier si le clic de la souris est bien compris dans la zone de l'un des deux rectangles qui nous sert de bouton jouer et quitter. Leurs valeurs sont représentées grâce à des variables de types int pour leurs coordonnées (x, y) mais également pour l'espace qu'ils occupent, donc leurs largeur et hauteur.

Variables utilisées :

- x_rectangle1, y_rectangle1 : coordonnées du bouton jouer (type int)
- x_rectangle2, y_rectangle2 : coordonnées du bouton quitter (type int)
- largeur_r1, Hauteur_r1 : largeur et hauteur du bouton jouer (type int)
- largeur_r2, Hauteur_r2 : largeur et hauteur du bouton quitter (type int)
- _xclic, _yclic : coordonnées du clic de la souris récupérées via `SourisPosition()` (type int)

Le choix des images

Lors de cette partie plusieurs valeurs vont être mobilisées notamment le même système que pour les boutons de l'écran d'accueil mais cette fois-ci ce seront les coordonnées des images qui remplaceront la largeur et la hauteur toujours de types int. Après avoir choisi l'image, 4 autres variables vont entrer en jeu, les 3 premières étant de types int et la dernière de type char :

Variables utilisées :

- x_rectangle1, y_rectangle1 : coordonnées de la première image (type int)
- x_rectangle2, y_rectangle2 : coordonnées de la deuxième image (type int)
- x_rectangle3, y_rectangle3 : coordonnées de la troisième image (type int)
- largeur_r1, Hauteur_r1 : largeur et hauteur de la première image (type int)
- largeur_r2, Hauteur_r2 : largeur et hauteur de la deuxième image (type int)
- largeur_r3, Hauteur_r3 : largeur et hauteur de la troisième image (type int)
- _xclic, _yclic : coordonnées du clic de la souris (type int)
- lignes : stocke le nombre de lignes choisi par l'utilisateur, débute à 0 (type int)
- colonnes : stocke le nombre de colonnes choisi par l'utilisateur, débute à 0 (type int)
- image : pointe vers l'image que l'on a choisie précédemment (type char*)

Les valeurs lignes et colonnes qui débutent à 0 vont stocker les valeurs en fonction de où l'utilisateur cliquera. Leurs valeurs respectives seront ensuite transmises à la fonction découpage pour qu'elle sache quel type de découpage effectuer.

La variable image de type char pointe vers l'image que l'on a choisie précédemment.

Les prémices du jeu

Pour la modélisation du puzzle nous avons utilisées des tableaux multidimensionnels de types int. Le premier nous sert de modèle et le deuxième nous permet de jouer. Pour remplir le tableau nous avons utilisées des boucles imbriquées avec les variables i et j de types int qui travaillent avec les variables l_tuile, h_tuile, num, numTile toutes de types int pour dans un premier temps découper l'image puis stocker les tuiles découpées dans les deux tableaux et dans un second afficher les deux tableaux à des endroits différents l'un de l'autre.

Variables utilisées :

- modele[8][8] : tableau représentant l'état résolu du puzzle, contient les valeurs 0, 1, 2, 3... dans l'ordre (type int)
- joueur[8][8] : tableau représentant l'état actuel du puzzle joué, contient les valeurs mélangées avec -1 pour la tuile blanche (type int)
- i, j : variables de boucle pour parcourir les tableaux (type int)
- l_tuile : largeur d'une tuile en pixels, calculée par $\text{img_L} / c$ (type int)
- h_tuile : hauteur d'une tuile en pixels, calculée par $\text{img_H} / l$ (type int)
- num : compteur pour numéroter les tuiles de 0 à $(l*c - 1)$ (type int)
- numTile : valeur stockée dans une case du tableau, représente l'identifiant de la tuile (type int)
- l : nombre de lignes du puzzle, transmis depuis le choix de l'utilisateur (type int)
- c : nombre de colonnes du puzzle, transmis depuis le choix de l'utilisateur (type int)
- img_L, img_H : dimensions de l'image originale en pixels (type int)
- src_x, src_y : coordonnées source pour copier une zone de l'image (type int)
- dst_x, dst_y : coordonnées destination pour afficher une tuile (type int)

Pendant le jeu

Variables supplémentaires utilisées pendant le jeu :

- nombre_coups : compteur de mouvements effectués, initialisé à 0 (type static int)
- centre_x, centre_y : coordonnées pour centrer le puzzle à l'écran (type int)
- bl_i, bl_j : position de la tuile blanche dans le tableau joueur, trouvée en cherchant la valeur -1 (type int)
- x, y : coordonnées temporaires pour les calculs d'affichage et de détection de clic (type int)
- mouvement_effectue : indicateur booléen pour savoir si un mouvement a été fait (type int)
- touche : code de la touche clavier pressée (type int)

Explications algorithmes de mélanges :

Au lieu de mélanger les tuiles comme un deck de cartes, le programme simule des déplacements réels de la tuile blanche, exactement comme si un joueur bougeait les tuiles avec les flèches.

Pour mélanger le puzzle, nous n'échangeons pas les tuiles au hasard.

À la place, nous déplaçons la case blanche comme si un joueur jouait réellement.

Le programme :

1. Trouve la position de la tuile blanche.
2. Exécute un grand nombre de déplacements aléatoires :
 - a. Haut
 - b. Bas
 - c. Gauche
 - d. Droite
3. Chaque déplacement est vérifié pour éviter de sortir de la grille.
4. À chaque fois, la tuile blanche échange sa position avec la tuile voisine.

Par conséquent, le mélange est toujours solvable car le programme fait que des coups "légaux" comme si un humain jouait. Donc tous les coups fait par le mélange sont réversibles.

Conclusion personnelle :

William-James Tafok :

Pendant la réalisation de cette SAE j'ai pu dans un premier temps revoir tout ce que nous avons utilisé précédemment en cours et en parallèles répondre à certaines interrogations que je me posais, comme le fait que je ne voyais pas vraiment à quoi allez nous servir tout ce qu'on apprenait en cours mise a pars à lancer des programmes sur le terminale. Le fait d'avoir effectué quelque chose de concret avec une interfaces graphiques était donc pour moi une bonne expérience, même si j'ai au début fait face à quelque difficulté. Tels que la résolution de bugs qui nous poussait à chercher la source de ces derniers, la réalisation de certaines fonctions, car bien que j'eusse joué au jeu, puis pris des notes sur comment le réaliser la mise en place n'était pas aussi évidentes que sur le papier. J'ai eu également au début des difficulté avec l'utilisation de la bibliothèque qui nous avait été fournis pour réaliser ce projet tels que l'utilisation de la fonction Copier Zones. Sinon comme avis global je dirais que c'était plaisant quand ça marchait du premier coup, ce qui était rarement le cas.

Théo Gobé :

Durant la SAE j'ai eu l'occasion d'apprendre pas mal de chose. Parmi ces choses, il y a le fait de s'organiser avant même d'écrire une seule ligne de code. Par s'organiser j'entends se répartir les tâches, faire un plan de ce que l'on va faire tel que

l'arborescence des appels de fonction, une idée d'a quoi va ressembler le jeu et etc.... J'ai aussi appris à gérer plusieurs fichiers dédié à un même programme notamment avec le makefile. J'ai aussi rencontré des problèmes tels que l'utilisation de git. Je pense d'ailleurs que c'est ce qui m'a le plus pris du temps, ta tête et que c'est là où j'ai le plus appris. A part git, j'ai pu faire face à des petits problèmes tels que des images qui ne chargent pas, des noms de fichier mal écrit dans le makefile et plein de petite erreur comme ça. En gros problème chronophage, il y a eu l'écran de victoire qui était compliquer à faire surtout la fonction qui vérifie la victoire dû au fait de finir le jeu à chaque test. Cependant malgré tout ça j'ai quand même apprécié ce projet et savourer encore plus d'avoir un jeu fonctionnel et fluide car on l'a bien optimisé et le plaisir d'avoir fini le projet n'a qu'été décupler.

FIN